

Pepe - El Arquero.

Ingeniería de Videojuegos.





CONTENIDO

Enunciado.....	2
Pepe – El Arquero.....	3
Programación y proceso de creación.....	3
Proceso de creación.....	3
Programación.....	4
Aplicación de los patrones.....	4
Fuentes de los Sprites utilizados.....	5



ENUNCIADO.

La práctica de la asignatura de Ingeniería de videojuegos consiste en la creación de un videojuego empleando el motor de videojuegos Unity Engine™ y aplicando las técnicas de desarrollo y patrones estudiados durante la asignatura.

Instrucciones:

- **Qué:** Realizar un videojuego, que puede basarse en los diferentes ejercicios realizados y disponibles en el aula virtual. El videojuego puede ser un videojuego de plataformas, o cualquier otro género, en 3D o en 2D.
- **Cómo:** Debe realizarse empleando Unity Engine™ y scripting C#.
- **Cuándo:** Debe entregarse con fecha límite el día anterior al examen de la parte final de la asignatura, antes de las 23:59 horas. Consultar fechas en el aula virtual, dentro del apartado Evaluación.
- **Quién:** Debe realizarse en grupos de 3 personas. Los grupos deberán inscribirse en el punto habilitado dentro del aula virtual, dentro del apartado Evaluación.
- **Cuánto:** La nota de la práctica tendrá en cuenta:
 - El cumplimiento de los requisitos.
 - Las decisiones y técnicas de diseño software adoptadas en la creación del videojuego.
 - El número y uso adecuado de patrones de desarrollo empleados en el videojuego.
 - La originalidad, dificultad e interés puestos en la práctica.

Requisitos del juego:

Los requisitos mínimos que debe cumplir el videojuego son:

- Se debe crear un videojuego con, al menos, un nivel. El objetivo es libre, recoger ítems, llegar a un determinado punto del escenario, pasar por una serie de checkpoints, ...
- El videojuego debe contar con un menú de inicio, que permita acceder al juego o salir, y con un menú de pausa.
- El videojuego debe mostrar en algún punto visible los nombres de los autores.
- Se deben emplear algunos o todos los patrones de desarrollo vistos durante la asignatura, los más adecuados a las funcionalidades que desarrollemos en nuestro videojuego. El número mínimo de patrones a desarrollar es 2.

Cumplir con estos requisitos mínimos (incluida la forma de entrega de la práctica que se cuenta a continuación) asegura la nota mínima para aprobar la práctica, no la máxima. Como se comenta en las instrucciones, la valoración de la práctica tendrá en cuenta las decisiones técnicas tomadas en el desarrollo del videojuego, la cantidad de patrones empleados y el uso que se haga de ellos, o la originalidad de la práctica.

No es importante para la evaluación el arte o la temática del juego, pueden emplearse además de los assets entregados en la plantilla, cualesquiera otros (sonidos, modelos, texturas) que se quiera, propios o de terceros.

No hagáis copia/pega de código. Por supuesto que podéis ver tutoriales, blogs, hacer consultas en stackoverflow (cuidado con las respuestas, no siempre son correctas o adecuadas), pero escribid siempre vuestro propio código. No se contará para la nota ningún bloque de código o funcionalidad que no sea original de los autores. Por ejemplo, el personaje que se facilita en la plantilla incluye un script para gestionar su movimiento, adaptado a sus animaciones. Este código, evidentemente no cuenta para la valoración.

En caso de emplear código o assets de terceros, debéis rendir crédito a sus autores originales. En el primer caso vale con un comentario al principio del archivo o delante de la función o método empleado. En el segundo, lo mejor es incluir un archivo de texto conteniendo los créditos en la carpeta de recursos donde se almacene el asset.



PEPE – EL ARQUERO.

PROGRAMACIÓN Y PROCESO DE CREACIÓN.

PROCESO DE CREACIÓN.

Para comenzar a crear el videojuego, hicimos un brain storming de ideas sobre qué tipo de videojuego queríamos realizar (plataformas, RPG, etc.), y, una vez decidido, sobre qué podía tratar.

Por otro lado, era importante pensar qué patrones podían ser interesantes de implementar en el juego, y de qué forma podíamos hacerlo, por lo que pusimos en común elementos y funcionalidades que nos gustaría que incluyese, y los relacionamos con los patrones impartidos y aprendidos en clase.

REPARTO DE TAREAS.

Al ser un trabajo en grupo, hemos trabajado de forma conjunta, con lo que todos hemos trabajado en todo.

Aún así, cada uno se ha centrado más en algunos patrones, según las mecánicas que se estaban introduciendo. En este caso, Rodrigo se ha centrado más en desarrollar correctamente los patrones Object Pool y Prototype, además del State, mientras que Enrique se ha centrado en patrones como el Observer, State y Dirty Flag. Por otro lado, Sara ha implementado el patrón Component y ha servido como apoyo y ayuda en el resto de las implementaciones de patrones y diseños de menús.

PROGRAMACIÓN.

PATRONES EMPLEADOS.

Se han empleado los patrones: Game Loop, Component, Observer, Dirty Flag, State, Prototype y Object Pool.

APLICACIÓN DE LOS PATRONES.

Patrón.	Aplicación.
Gameloop.	El Gameloop no ha sido implementado con código, pero se ha hecho uso del Gameloop por defecto de Unity utilizando la función FixedUpdate para actualizar las físicas de los distintos objetos y personajes de la escena. Por otro lado, también se ha utilizado para el movimiento de la cámara mediante la función LateUpdate.
Component.	Este patrón Component se ha empleado para desacoplar las funciones del personaje creando varios componentes para los distintos comportamientos de este, como el salto, disparar, etc.
Observer.	El patrón Observer se ha utilizado para poder recoger monedas, interactuar con el jugador si se choca con algo que le hace daño, y, por último, para las vidas del enemigo y las del personaje principal
Dirty Flag.	Se emplea el patrón Dirty Flag para poder guardar el estado del juego en el check point dispuesto en el escenario (las rocas).
State.	El patrón State se ha empleado para implementar el menú de pausa y para definir el comportamiento del enemigo, el cual, si no detecta al personaje principal, patrulla, y si lo ve, lo persigue.
Prototype.	Este patrón se ha empleado para usar varias instancias de las monedas, flechas y pinchos (rocas picudas).
Object Pool.	Se ha usado este patrón Object Pool para reutilizar las flechas lanzadas por el personaje principal.

FUENTES DE LOS SPRITES UTILIZADOS.

Todos los Sprites que aparecen en el juego son ajenos, descargados de la página web <https://lyaseek.itch.io> (Sprites de personajes) y <https://craftpix.net/categorys/sprites/> (suelo y paredes, además de los elementos que aparecen en el escenario, como monedas, rocas y pinchos).